

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/387952132>

Benchmarking YOLOv8 for Optimal Crack Detection in Civil Infrastructure

Conference Paper · January 2025

CITATIONS

0

READS

36

5 authors, including:



[Woubishet Zewdu Taffese](#)

Aalto University

53 PUBLICATIONS 1,074 CITATIONS

[SEE PROFILE](#)



[Ritesh Sharma](#)

The University of Tulsa

21 PUBLICATIONS 164 CITATIONS

[SEE PROFILE](#)



[Mohammad H. Afsharmovahed](#)

Missouri University of Science and Technology

7 PUBLICATIONS 15 CITATIONS

[SEE PROFILE](#)

Benchmarking YOLOv8 for Optimal Crack Detection in Civil Infrastructure

Woubishet Zewdu Taffese

Department of Civil, Architectural and Environmental Engineering,
Missouri University of Science and Technology, Rolla, MO, and 65409-6510
Email: wtcz8@mst.edu

Ritesh Sharma

Department of Civil, Architectural and Environmental Engineering,
Missouri University of Science and Technology, Rolla, MO, and 65409-6510
Email: rsr4z@mst.edu

Mohammad Hossein Afsharmovahed

Department of Civil, Architectural and Environmental Engineering,
Missouri University of Science and Technology, Rolla, MO, and 65409-6510
Email: maxnf@mst.edu

Gunasekaran Manogaran

Department of Civil, Architectural and Environmental Engineering,
Missouri University of Science and Technology, Rolla, MO, and 65409-6510
Email: gsgwp@mst.edu

Genda Chen

Department of Civil, Architectural and Environmental Engineering,
Missouri University of Science and Technology, Rolla, MO, and 65409-6510
Email: gchen@mst.edu

Word Count: 5904 words + 5 table (250+250+250+250+250 = 1250 words) = 7154 words

Submitted on November 19, 2024

ABSTRACT

Ensuring the structural integrity and safety of bridges is crucial for the reliability of transportation networks and public safety. Traditional crack detection methods are increasingly being supplemented or replaced by advanced artificial intelligence (AI) techniques. However, most of the models rely on two-stage target detection algorithms, which pose concerns for real-time applications due to their lower speed. While models such as YOLO (You Only Look Once) have emerged as transformative tools due to their remarkable speed and accuracy. However, the potential of the latest YOLOv8 framework in this domain remains underexplored. This study bridges that gap by rigorously evaluating YOLOv8's performance across five model scales (nano, small, medium, large, and extra-large) using a high-quality Roboflow dataset. A comprehensive hyperparameter optimization was performed, testing six state-of-the-art optimizers—Stochastic Gradient Descent, Adaptive Moment Estimation, Adam with Decoupled Weight Decay, Root Mean Square Propagation, Rectified Adam, and Nesterov-accelerated Adam. Results revealed that YOLOv8, optimized with Stochastic Gradient Descent, delivered exceptional accuracy and speed, setting a new benchmark for real-time crack detection. Beyond its immediate application, this research positions YOLOv8 as a foundational approach for integrating advanced computer vision techniques into infrastructure monitoring. By enabling more reliable and proactive maintenance of aging bridge networks, this work paves the way for safer, more efficient transportation systems worldwide.

Keywords: YOLO, Crack, Deep Learning, Model, Infrastructure, Bridge

INTRODUCTION

Ensuring the structural integrity and safety of bridges is essential for the smooth operation of transportation networks and the protection of human lives. Regular maintenance and timely detection of structural defects, such as cracks, are crucial to preventing catastrophic failures. This need is increasingly urgent, given that much of our critical infrastructure—such as bridges, highways, and buildings—was constructed several decades ago and is beyond its intended lifespan. For example, the 2021 America's Infrastructure Report Card by the American Society of Civil Engineers (ASCE) highlights that 42% of bridges in the United States are over 50 years old, with approximately 7.5% (46,154) classified as structurally deficient (1). To ensure the safety of bridges, the National Bridge Inspection Standards (NBIS) mandate inspections every two years (2).

One major type of defects on concrete bridges is surface cracks, which can lead to functionality issues and even bridge failure if not addressed early enough. Recent technological advancements have led many scholars to turn to artificial intelligence (AI) for its speed and high accuracy in crack detection and segmentation applications. Within AI, Deep Learning (DL), a branch of Machine Learning (ML), is commonly employed in crack detection studies. For example, Kun et al. (2022) proposed Deep Bridge Crack Classification (DBCC)-Net, a two-stage classification-based DL network (3). Similarly, Li et al. (2022) introduced the Dense Boundary Refinement Network (DBR-Net) for automatic bridge crack detection (4), while Li et al. (2023) advanced detection techniques by implementing the Faster R-CNN algorithm for the same purpose (5). Lu et al. (2022) developed an improved Single Shot MultiBox Detector (ISSD) for bridge crack detection (6). Other notable contributions include CrackHAM, introduced by He and Lau (2024), a U-Net-based model specifically designed for automatic crack detection (7). Zoubir et al. (2024) presented a framework for pixel-level concrete bridge crack detection combining U-Net, Gabor filters, and Convolutional Block Attention Modules to achieve pixel-level detection (8). Additionally, Matarneh et al. (2024) conducted a comprehensive evaluation of pre-trained Convolutional Neural Network (CNN) architectures for pavement crack detection and classification, identifying the architecture that delivered optimal results (17). Zhang et al. (2024) presented an enhanced U-Net model incorporating VGG16 architecture, Up_Conv modules, and channel attention layers to boost detection capabilities over standard U-Net models (18). Despite their high prediction performance, many of these models rely on two-stage architectures that involve region proposal generation followed by bounding box refinement. While effective, this approach poses challenges for real-time applications due to its relatively slower processing speed (19).

YOLO (You Look Only Once) (9) employs a single neural network to predict bounding boxes and class probabilities directly from images in one evaluation, addressing the limitations observed in the two-stage target detections. YOLO is designed to enhance both speed and accuracy with its single-stage detection algorithm, making it suitable for real-time applications. YOLO has evolved through several model versions, and recently, researchers have examined its application for crack detection. For instance, Gong et al. (2023) explored key factors affecting the performance of DL models in automated pavement crack detection, emphasizing the importance of model selection, dataset size, and annotation consistency, with YOLO models outperforming alternatives (10). Zhang et al. (2022) applied YOLOv4 to automate bridge crack detection (11), while Liu et al. (2023) used YOLOv5, enhancing model training with artificial samples for rapid crack detection (12). Su et al. (2024) introduced YOLO v3 for detecting bridge damage in complex backgrounds (13) and Yang et al. (2024) developed an improved YOLOv7 model tailored for bridge crack detection (14).

Building on these advancements, Su et al. (2024) created MOD-YOLO to improve crack detection in infrastructure (20, 21). Tang et al. (2024) enhanced YOLOv5s by refining anchor points, incorporating Convolutional Block Attention Modules (CBAM) for detecting small crack detection, and Complete Intersection over Union Loss (CIoU_Loss) for improved accuracy (22). Liu & Liu (2024) also further optimized YOLOv5s, achieving higher accuracy and faster detection times (23). Recent efforts have focused on YOLOv8 for crack detection. For example, Xiong et al. (2024) introduced the YOLOv8-GAM-Wise-

IoU model for automated bridge crack detection (15), followed by their YOLOv8-AFPN-MPD-IoU model, enhancing detection precision for bridge cracks(24). Additionally, Li et al. (2024) introduced DCNA-YOLO, a YOLOv8-based approach for detecting superficial defects in concrete bridges (16).

Recent publications have explored YOLOv8, yet none have concentrated on the essential role of hyperparameter tuning procedure—a critical factor for achieving optimal predictions with this advanced model. Addressing this gap, this study pioneers the application of YOLOv8 specifically for concrete crack detection, a domain where its potential has not been thoroughly explored. With its innovative architecture, lightweight design, and real-time image processing capabilities, YOLOv8 offers exceptional accuracy and efficiency for object detection tasks, making it an ideal choice for automated bridge crack detection. Comprehensive evaluations were conducted for all scaled versions of YOLOv8—nano, small, medium, large, and extra-large—using a high-quality, diverse dataset curated and annotated via Roboflow (25), a platform renowned for optimizing computer vision models. By integrating six distinct optimizers, the most accurate combinations were identified, followed by rigorous hyperparameter tuning to refine performance and establish benchmark standards for crack detection accuracy. This research not only demonstrates YOLOv8's superior performance in a novel application but also provides a robust framework for leveraging advanced computer vision techniques in infrastructure monitoring.

Overview of YOLOv8

YOLO, introduced by Redmon et al. (9), employs CNN to detect objects within images by predicting bounding boxes and their associated probabilities. The process starts with partitioning the image into a grid of $N \times N$ cells, with each cell responsible for predicting a certain number of bounding boxes. For each bounding box, YOLO predicts coordinates, dimensions, and a confidence score, which indicates both the likelihood that the box contains an object and the accuracy of its placement. Over the years, YOLO has evolved through several iterations, each enhancing its speed and accuracy. YOLOv8 (8), not only surpasses its predecessors but also emphasizes real-time application efficiency. YOLOv8, developed by Ultralytics (26), represents state-of-the-art real-time object detection in the YOLO series, offering significant improvements in speed, accuracy, and user-friendliness compared to earlier versions (9, 27–31). YOLOv8 is available in five scaled models: YOLOv8n (nano), YOLOv8s (small), YOLOv8m (medium), YOLOv8l (large) and YOLOv8x (extra-large). Each variant differs in architecture, intended use cases, and trade-offs between speed and accuracy. The architectural variation, including the number of parameters, layers, and input image sizes, influences the performance and efficiency of these scaled models. A detailed summary of YOLOv8 variants is presented in **Table 1**.

TABLE 1 Details of YOLOv8 variants analyzed (32)

Model	Size (pixels)	Params (M)	FLOPs (B)
YOLOv8n	640	3.2	8.7
YOLOv8s	640	11.2	28.6
YOLOv8m	640	25.9	78.9
YOLOv8l	640	43.7	165.2
YOLOv8x	640	68.2	257.8

YOLOv8 Architecture

The architecture of YOLOv8 comprises three primary components: the backbone network, the neck and the head as illustrated in **Figure 1**. The process begins with standardizing the input images by resizing them to a uniform dimension and normalizing pixel values to the range of 0 to 1. This standardization ensures consistency across input data, minimizing discrepancies caused by varying camera parameters and lighting conditions. The standardized images are then fed into the backbone network, a modified version of the Darknet architecture known as CSPDarknet53. This modification enhances the network's learning capacity and efficiency by learning features from the input images at multiple levels of abstraction. The

features extracted by the backbone are subsequently passed through the neck component which consists of a Feature Pyramid Network (FPN), also known as Path Aggregation Network (PANet). PANet is crucial for effectively handling objects at various scales by aggregating the features extracted at different scales and improving the detection of objects of different sizes. Finally, the processed features are directed to the head component, which generates the final predictions. The head performs bounding box regression to predict the bounding boxes around the detected objects and assigns class probabilities. Each grid cell predicts multiple bounding boxes along with confidence scores and the probabilities for each class of the detected objects.

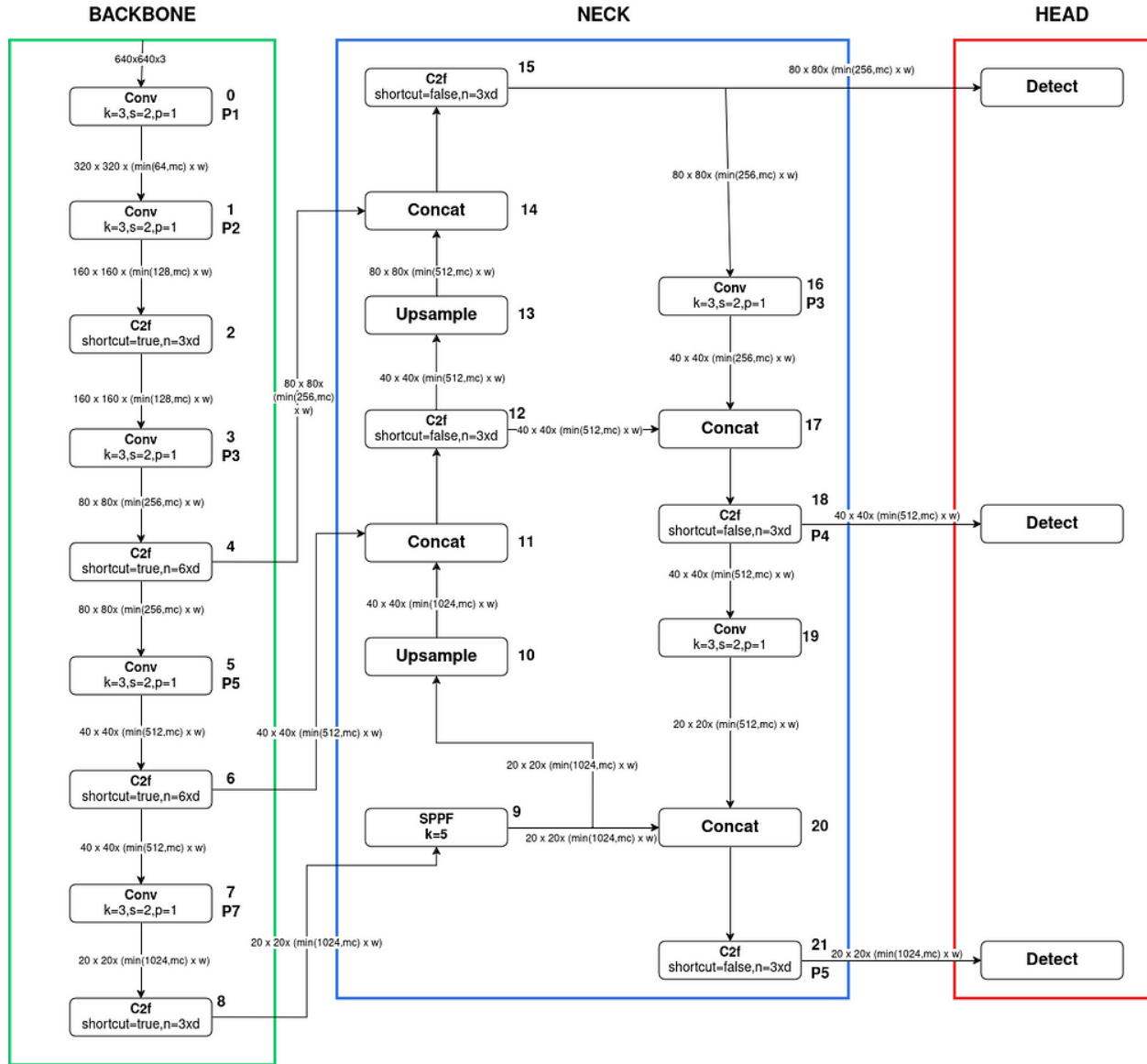


Figure 1 YOLOv8 architecture (33)

Central to YOLOv8's robust detection capabilities are metrics and algorithms such as Intersection over Union (IoU) and Non-Max Suppression (NMS). The IoU metric, defined in **Equation 1**, evaluates the overlap between predicted bounding boxes and ground truth boxes, forming the basis for confidence scoring. Predictions with confidence scores above a certain threshold are retained, while NMS eliminates

redundant bounding boxes, ensuring only the most accurate detection per object. Additionally, YOLOv8 integrates IoU into its loss function, refining predictions and improving detection performance.

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}, \quad (1)$$

where, the “*Area of Overlap*” is the intersection between the predicted bounding box and the ground truth bounding boxes, while the “*Area of Union*” refers to the combined area covered by both the predicted bounding box and the ground truth bounding box.

The confidence score is defined as the likelihood that an object is inside the predicted box and is computed as shown in **Equation 2**:

$$\text{Confidence Score} = \text{Pr}(\text{Object}) \times IoU. \quad (2)$$

These confidence scores help filter out predictions with low confidence scores or poor bounding box accuracy. For each predicted bounding box, the class probability represents the likelihood that a particular type of object is present in the predicted bounding box, among all possible objects. The class probability is computed as shown in **Equation 3**:

$$\text{Pr}(\text{Class} = x | \text{Object}) = \frac{e^{z_x}}{\sum_{j=1}^c e^{z_j}}. \quad (3)$$

where z_x is the output score for class x , z_j represents the scores for all classes $j = 1, 2, \dots, c$, and c is the total number of classes.

MATERIALS AND METHODS

This section provides an overview of the dataset used, including their sources and relevance. An outline of implementation framework and configuration details for model training is presented. The selected optimizers for training, along with their parameters and impact, and the performance evaluation metrics employed to assess the model effectiveness are described.

Dataset description

The dataset utilized in this study is sourced from Roboflow, a comprehensive platform renowned for its capabilities in managing, annotating, pre-processing, and augmenting datasets within the computer vision domain. This dataset comprises 1170 images that encompass a diverse array of concrete crack types with varying patterns, background textures, and severity levels, as illustrated in Figure 2. These images were then annotated using the same platform, ensuring consistency and accuracy throughout the dataset preparation process.



Figure 2. Example of concrete crack images considered in the dataset

Implementation details

Experiments were conducted on a system with a 13th Generation Intel Core i9 processor, 64 GB of RAM and NVIDIA RTX 4070 GPU (12 GB memory). Additional computational power was leveraged via Google Colab Pro+, featuring an NVIDIA V100 GPU and up to 52 GB high RAM during 24-hour sessions. Various scales of YOLOv8 models were applied to the dataset, including the YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x versions. These models differ in architectural complexity, intended use cases, and tradeoffs between speed and accuracy.

Regarding optimizers, six different algorithms were employed: Stochastic Gradient Descent (34), Adaptive Moment Estimation (Adam) (35), Adam with decoupled weight decay (36), Root Mean Square Propagation (37), Rectified Adam (38), and Nesterov-accelerated Adam (39). These optimizers compute gradient descent and minimize the loss functions to determine optimal values during the model's training process.

Stochastic Gradient Descent (SGD)

SGD (34), a prevalent used optimization technique that updates weights using a randomly selected batch of data in each iteration. The iterative formula for SGD is given in **Equation 4**:

$$\theta_{t+1} = \theta_t - \alpha_t \nabla f(\theta_t), \quad (4)$$

where θ_t is weight, $f(\theta_t)$ is a loss function along with its gradient $\nabla f(\theta_t)$, and α_t is the learning rate.

The randomness in batch selection introduces noise, helping escape local minima and improving convergence and generalization. However, an improper learning rate can cause issues—too high may lead to divergence, while too low can slow down convergence.

Root Mean Square Propagation (RMSProp)

RMSProp, introduced by Geoffrey Hinton (37), is a gradient-based optimization algorithm that adapts the learning rate using the moving average of squared gradients, enabling faster convergence and better handling of sparse or noisy gradients. The moving average of squared gradients is computed as described in **Equation 5**:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla f(\theta_t)^2, \quad (5)$$

where v_t is the moving average of squared gradients, v_{t-1} is the moving average of squared gradients from the previous time step $t - 1$, and β is the decay factor, and $\nabla f(\theta_t)^2$ is the square gradient.

The weights are then updated as in the **Equation 6**:

$$\theta_{t+1} = \theta_t - \frac{\alpha_t}{\sqrt{v_t + \epsilon}} \nabla f(\theta_t), \quad (6)$$

where ϵ is a small constant added for numerical stability.

Adaptive Moment Estimation (Adam)

Adam(35) combines features of RMSprop and Momentum to compute adaptive learning rates for each parameter. It maintains an exponentially decaying average of past gradients and past squared gradients.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f(\theta_t), \quad (7)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \nabla f(\theta_t)^2, \quad (8)$$

where β_1 and β_2 represents decaying factor, m_t is the moving factor and v_t is the moving average of the squared gradient.

At the initial time step, m_t and v_t are set to zero which makes the update biased towards zero especially when the decay rates are small. To counteract such biases, the first and second moment are bias corrected, resulting in $\hat{m}_t$ and $\hat{v}_t$, as shown in the **Equations 9-10**:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}. \quad (9)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (10)$$

Equation 11 provide the update rule for Adam.

$$\theta_{t+1} = \theta_t - \frac{\alpha_t \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}. \quad (11)$$

Adam's key feature is its use of the exponential decaying average of past squared gradients to estimate both the momentum and the second moment of the gradient, thus, customizing each parameter's learning rate based on gradient history and adjusts the direction of the update. Various variants of Adam optimizer exist, and several of them have been utilized, as discussed below.

Adam with decoupled weight decay (AdamW)

AdamW (36) modifies the Adam algorithm by decoupling learning rate and weight decay, applying weight decay directly to the weights. This separation improves optimization stability and generalization. This separation enhances training stability and generalization. The update rule for AdamW is given by **Equation 12**:

$$\theta_{t+1} = \theta_t - \frac{\alpha_t \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \lambda \theta_t, \quad (12)$$

where λ is the weight decay and ϵ is the small constant which is used for numerical stability.

Rectified Adam (RAdam)

RAdam (38) addresses instability from aggressive learning rates in the early training phase by introducing a rectification term R to stabilize the adaptive learning rate. The rectification term is computed as described by **Equation 13**:

$$R = \sqrt{\frac{(1 - \beta_2^t) \max(1, t - \frac{4}{\beta_2 - 1})}{\beta_2^t}}. \quad (13)$$

The update rule for RAdam is given as **Equation 14**:

$$\theta_{t+1} = \theta_t - \frac{\alpha_t \hat{m}_t}{\sqrt{\hat{v}_t + R}}. \quad (14)$$

Nesterov-accelerated Adam (NAdam)

Nadam (39) combines the Adam optimizer with the Nesterov Accelerated Gradient (NAG), enhancing adaptive learning rates and gradient estimation accuracy for improved convergence and performance. NAG computes the gradient at a projected update of the parameters, rather than at their current values. The lookahead gradient is given by **Equation 15**.

$$\tilde{\nabla} f(\theta_t) = \nabla f\left(\theta_t - \alpha_t \frac{m_t}{\sqrt{v_t + \epsilon}}\right). \quad (15)$$

The update rule for NAdam can then be written as **Equation 16**:

$$\theta_{t+1} = \theta_t - \frac{\alpha_t (\beta_1 \tilde{\nabla} f(\theta_t) + (1 - \beta_1) \nabla f(\theta_t))}{\sqrt{\hat{v}_t + \epsilon}}. \quad (16)$$

The following steps outline the model training process for crack detection using YOLOv8. Each phase is designed to optimize the model's performance through careful selection of pre-trained weights, hyperparameter tuning, and comprehensive optimization techniques.

Step 1: Initialization and Training - The process begins by loading the pre-trained weights of YOLOv8, providing a strong foundation for the subsequent stages. During this phase, the model is trained on a crack detection-specific dataset, leveraging the pre-trained weights for transfer learning. Default parameter settings are maintained to evaluate which YOLOv8 variant and optimizer combination delivers the highest accuracy in crack detection.

Step 2: Hyperparameter Tuning - Next, hyperparameter tuning is performed to refine the model selection among the best YOLOv8 variants and optimizers identified in Step 1. Due to computational resource constraints, an exhaustive search is avoided. Instead, the One Factor at a Time (OFAT) (40), approach is adopted, systematically adjusting one hyperparameter at a time while keeping others constant.

This targeted approach narrows the choices to a specific YOLOv8 variant and optimizer, laying the groundwork for more in-depth optimization in the next step.

Step 3: Comprehensive Optimization - The final phase involves a thorough exploration of hyperparameter configurations using random search. This step builds on the insights from Step 2, focusing on key hyperparameters to fine-tune the selected YOLOv8 model and optimizer combination. The goal is to identify the best possible setup for achieving accurate and robust crack detection.

Model evaluation

This research study adopts the evaluation metrics of precision, recall, and mean average precision (mAP) to scrutinize the capabilities of the trained YOLOv8 crack detection models.

Precision measures the proportion of true positive (TP) samples among the samples predicted to be positive (41). TPs are those that were actually positive and were predicted to be positive. Precision is used as a performance metric when the goal is to limit the number of false positives (FP), which are samples predicted to be positive but were actually negative. Precision is calculated as shown in **Equation 17**:

$$Precision = \frac{TP}{TP+FP}. \quad (17)$$

Recall, on the other hand, measures the proportion of true positive samples that are correctly identified by the model (41). It is used as a performance metric when identifying all positive samples is crucial, particularly to avoid false negatives (FN), which are samples that were actually positive but were incorrectly predicted to be negative. Recall is calculated using the formula shown in **Equation 18**:

$$Recall = \frac{TP}{TP+FN}. \quad (18)$$

Mean Average Precision (mAP) is derived from the concepts of precision and recall and is extensively used to measure the performance of object detection models. The area under the precision-recall curve is calculated, which gives the Average Precision (AP) for each class. The mean Average Precision (mAP) is then computed by averaging the APs across all classes. The formula shown in **Equation 19** is used to calculate mAP (32).

$$mAP = \frac{1}{n} \sum_{i=1}^n AP_i. \quad (19)$$

RESULTS AND DISCUSSION

Following the training of various YOLOv8 model variants using default values, a comparative evaluation was conducted to assess their performance based on precision, recall, and the mean Average Precision at 50% IoU threshold (mAP50). This threshold is commonly employed due to its simplicity in computation and interpretation, making it a suitable metric for initial model evaluations (32).

Table 2 presents the precision, recall and mAP50 scores for a range of YOLOv8 models, including YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x, each trained with six distinct optimization algorithms (SGD, Adam, AdamW, RMSprop, NAdam, and RAdam). The higher the precision, recall, and mAP50 scores, the more accurate the model is in detecting objects, with fewer false positives and false negatives, thus demonstrating superior performance in identifying and localizing objects. The results indicate that the YOLOv8m model generally achieves the highest scores across all three metrics, indicating that it outperforms the other YOLOv8 variants in terms of accuracy. Additionally, it is observed that within the YOLOv8 series, the optimizers SGD, Adam, and AdamW consistently yield better accuracy compared to the other optimizers.

TABLE 2 Performance evaluation results of each YOLOv8 variant on the crack dataset

YOLOv8 variant	Optimizer	Precision	Recall	mAP50	Training time (hours)
YOLOv8n	AdamW	0.88	0.701	0.809	0.282
	SGD	0.859	0.713	0.811	0.182
	Adam	0.828	0.645	0.732	0.182
	RMSprop	0.00814	0.511	0.0113	0.277
	NAdam	0.837	0.66	0.744	0.286
	RAadam	0.875	0.655	0.762	0.182
YOLOv8s	AdamW	0.875	0.701	0.814	0.493
	SGD	0.862	0.768	0.846	0.209
	Adam	0.843	0.63	0.721	0.215
	RMSprop	0.00316	0.276	0.00292	0.208
	NAdam	0.79	0.634	0.697	0.491
	RAadam	0.86	0.648	0.743	0.214
YOLOv8m	AdamW	0.864	0.707	0.814	0.924
	SGD	0.85	0.788	0.85	0.323
	Adam	0.73	0.615	0.633	0.918
	RMSprop	0.00303	0.432	0.0154	0.322
	NAdam	0.662	0.581	0.614	0.913
	RAadam	0.837	0.641	0.734	0.923
YOLOv8l	AdamW	0.858	0.685	0.774	0.441
	SGD	0.896	0.745	0.84	0.43
	Adam	0.182	0.285	0.0445	0.441
	RMSprop	0.000317	0.0598	0.00305	0.406
	NAdam	0.207	0.323	0.139	0.444
	RAadam	0.844	0.625	0.715	0.444
YOLOv8x	AdamW	0.866	0.661	0.491	2.027
	SGD	0.879	0.768	0.844	0.61
	Adam	0.65	0.548	0.579	0.619
	RMSprop	0.00173	0.405	0.0198	0.608
	NAdam	0.134	0.429	0.137	1.973
	RAadam	0.832	0.617	0.705	0.62

The OFAT technique involves systematically varying one hyperparameter while holding others constant, allowing for an isolated assessment of each hyperparameter's impact on model performance, offering valuable insights into their individual significance. By identifying the most critical factors, OFAT helps refine the search space and narrow down the range of potential values for each parameter.

In this study, the OFAT method is employed to analyze several key hyperparameters, including epoch, batch size, learning rate, optimizer, momentum, and weight decay. These parameters were selected based on their established influence on model efficacy and performance, as documented in the literature. Specifically, learning rate, optimizer type, and epoch count are well-recognized for their crucial roles in model training dynamics. Additionally, weight decay and batch size, as recommended by the developers of YOLOv8, were included due to their anticipated impact on model performance. The systematic exploration

of these hyperparameters is detailed in **Table 3**, which lists the specific configurations used in the OFAT phase of this study.

TABLE 3 Hyperparameters configurations considered during the OFAT phase of tuning

Hyperparameters configuration sets	Hyperparameters Ranges				
	Epoch	Batch	lr0	momentum	Weight decay
H-1	50	32	0.1	0.9	0.001
H-2	75	64	0.01	0.8	0.005
H-3	100	64	0.001	0.7	0.01
H-4	100	64	0.002373	0.937	0.0004213

Table 4 presents the training, validation, and test performance of all models following OFAT hyperparameter tuning. The results reveal that YOLOv8m models utilizing the SGD optimizer exhibit robust performance across training, validation and testing phases. Among these, the YOLOv8m model with the H-4 hyperparameter configuration and SGD optimizer emerges as the top performer, surpassing all other tested configurations. Although the H-4 configuration has not yet been evaluated with Adam and AdamW optimizers, the SGD optimizer outperforms Adam and AdamW across the tested configurations (H-1, H-2, and H-3), establishing its dominance in this context. The identified optimal hyperparameter configuration sets provide a solid foundation for further fine-tuning. Moreover, **Table 2** underscores the remarkable efficiency of the YOLOv8m model when trained with the SGD optimizer, requiring significantly less training time compared to alternative configurations. This efficiency, coupled with the model's robust performance, highlights the practical advantages of this setup for time-sensitive and computationally demanding tasks.

TABLE 4 Performance of the three optimizers on training, validation and test sets

Optimizer	Hyperparameters configuration sets	Models Evaluation Metrics					
		mAP50 (Train)	No of Val Img	mAP50 (Val)	No of Test Img	mAP50 (Test)	Run time (hrs)
SGD	H-1	0.764	614	0.818	562	0.701	0.965
	H-2	0.944	614	0.882	562	0.792	1.353
	H-3	0.838	614	0.839	562	0.725	1.799
	H-4	0.962	614	0.876	562	0.805	1.791
Adam	H-1	0.299	614	0.297	562	0.291	0.957
	H-2	0.577	614	0.659	562	0.527	1.373
	H-3	0.705	614	0.793	562	0.725	1.828
AdamW	H-1	0.426	614	0.468	562	0.46	0.581
	H-2	0.729	614	0.79	562	0.664	0.758
	H-3	0.564	614	0.614	562	0.536	2.354

Following the completion of the OFAT analysis and a thorough evaluation of its results, the study transitioned to a random search focused on the hyperparameters identified as having the most significant impact on model performance. The primary objective was to explore whether any previously unexamined configurations could further enhance the model's capability in detecting cracks. Given that the OFAT analysis indicated optimal results with a batch size of 64 and a training duration of 100 epochs, these hyperparameters were fixed during random search. A log-uniform sampling method was employed for learning rate, weight decay, and momentum, allowing for effective sampling within the specified ranges. To achieve a comprehensive yet efficient exploration of the hyperparameter space, 20 repetitions of the random search were performed. This balance between thorough exploration and computational efficiency ensured a rigorous search while managing computational costs effectively. **Table 5** outlines the hyperparameters selected for experimentation during the random search, based on insights gained from the OFAT analysis. The table also displays the optimal hyperparameters identified through the random search.

TABLE 5 Ranges of hyperparameters for random search and the optimal configurations

Ranges of hyperparameters		Optimal hyperparameters
Dropout	0.18 – 0.37	0.29
Learning Rate	$1.177e^{-3} - 8.838e^{-3}$	$3.139e^{-3}$
Momentum	0.8 – 0.97	0.89
Weight Decay	$5.7413e^{-5} - 4.823e^{-4}$	$4.149e^{-4}$

Figure 3 depicts the loss values for box loss, classification loss, and distribution focal loss across each training epoch. It also shows precision, recall, mAP50, and mAP50–95 metrics after 100 training epochs. The YOLOv8m model achieved notable results, with an mAP50 of 0.957 and a mAP50–95 of 0.886 on the validation set, highlighting its ability to accurately detect cracks with high confidence. Box loss measures the difference between predicted and actual bounding box coordinates, class loss evaluates the accuracy of object classification, distribution focal loss reflects the network's performance in predicting probabilities for coordinates near target values. As these loss functions are minimized during training and validation, they indicate continuous improvement in crack detection accuracy. The figure also reveals a substantial increase in precision, recall, and mAP at the start of the training, with a gradual rise until convergence around 50 epochs. This trend highlights the model's progressive enhancement in crack detection performance with each epoch.

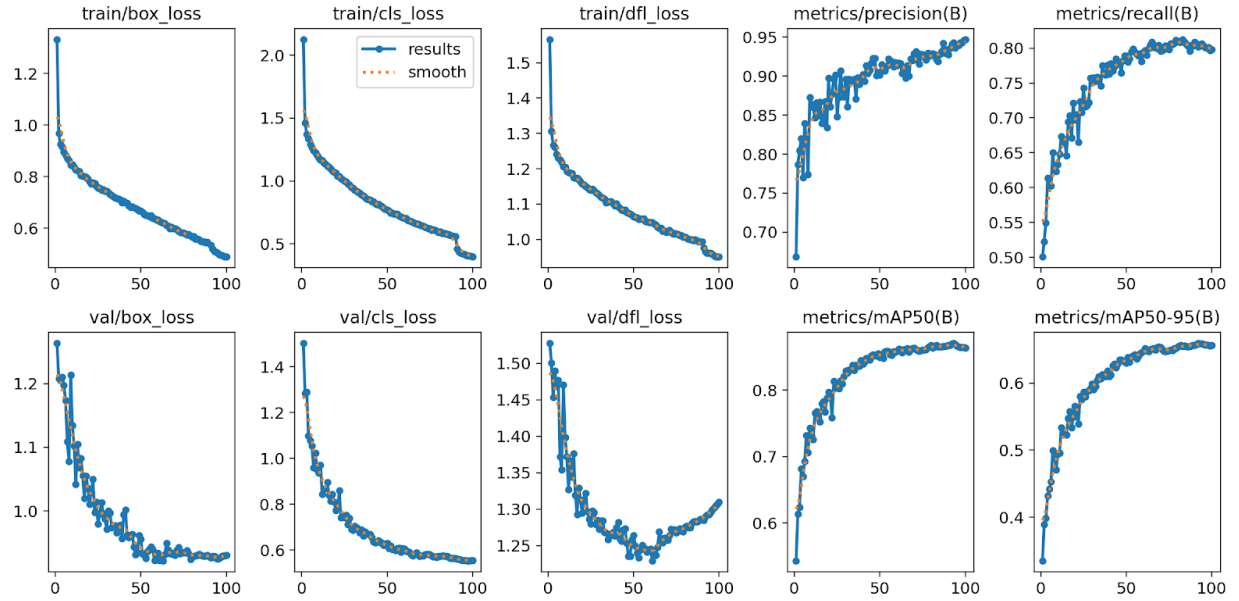
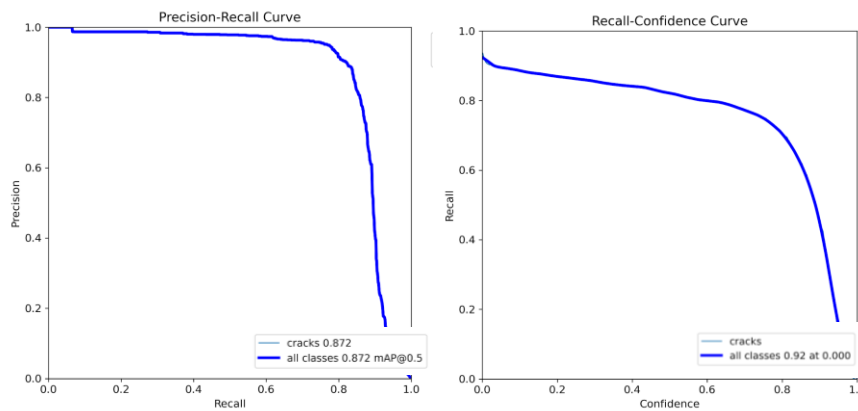


Figure 3 Training and validation performance of the model.

The prediction performance on the test dataset is evaluated using metrics, including the precision-recall curve, precision-confidence curve, recall-confidence curve, and F1-confidence curve, as illustrated in **Figure 4**. These metrics provide a comprehensive understanding of the model's ability to accurately detect and classify cracks within images. The precision-recall and F1-score curves illustrate the trade-off between precision and recall across various decision thresholds. The precision-confidence curve indicates that higher confidence scores are associated with higher precision values. A large area under the recall-confidence curve indicates high recall and a low rate of false negatives. The model's position at the top right corner of the precision-recall curve reflects a significant area under the curve, highlighting its effectiveness. Additionally, the F1-confidence curve demonstrates that the model achieved a maximum F1-score of 0.96. The detected cracks using the best model (YOLOv8m with the SGD optimizer) on the test dataset, along with the bounding boxes, are illustrated in **Figure 5**.



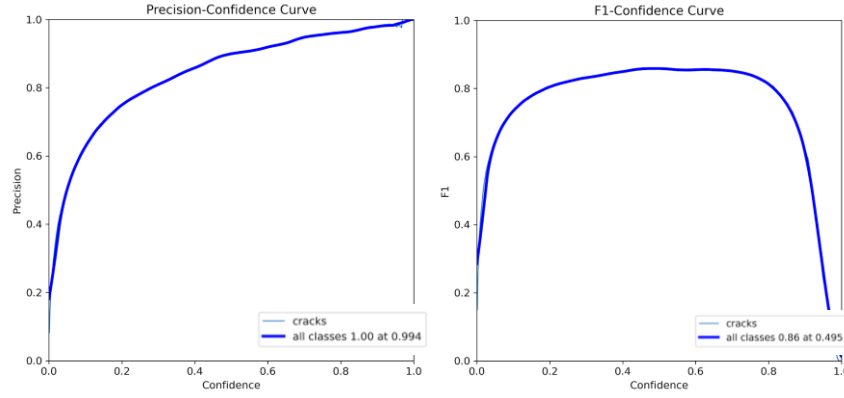


Figure 4 Performance evaluation curves

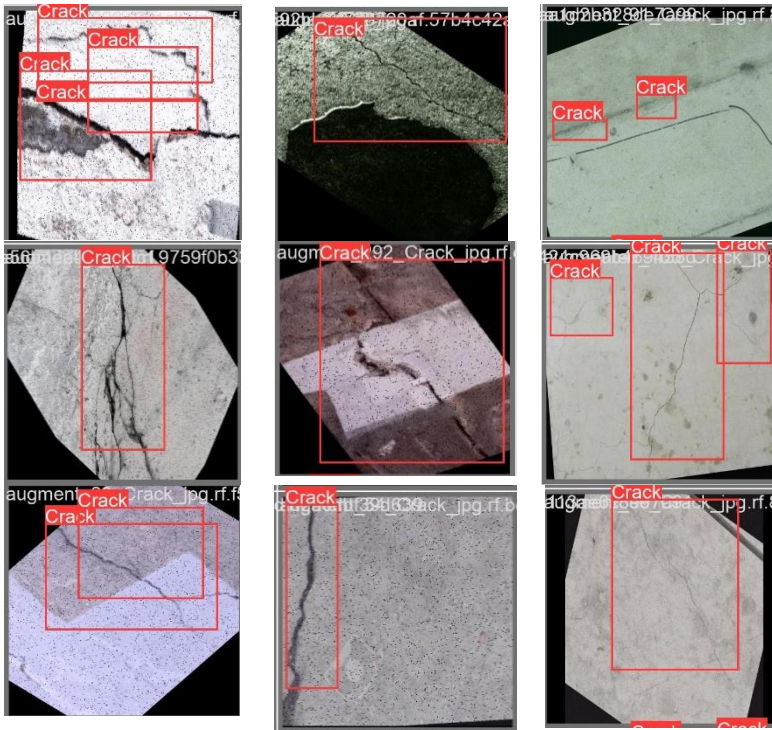


Figure 5 Detected cracks using the best performing model

The exhaustive examination of various YOLOv8 model variants, incorporating a range of optimizers and a comprehensive hyperparameter tuning process, has led to the identification of the most effective models in terms of both accuracy and speed for crack detection. This rigorous analysis provides a foundation for adopting optimal models in infrastructure inspection, a field that has increasingly benefited from the rapid advancements in remote sensing technologies for real-time monitoring.

The work presented not only highlights the best-performing models for crack detection but also sets a precedent for future research in this domain. However, it is important to acknowledge that the analysis was conducted using models trained on a relatively limited dataset, which may impact the repeatability of the results under varying conditions such as distance, lighting, and camera angles. Environmental factors like motion blur, occlusions, and background complexity further impact detection accuracy, underscoring the need for a robust dataset. To address these limitations and improve the robustness of the presented models, authors plan to expand the dataset by collecting crack images from a variety of bridges and other

infrastructure, captured under diverse environmental factors. This enriched dataset will encompass diverse types of cracks and will be augmented using Generative Model based techniques, which have shown promise in several studies (42–44). With this expanded and diverse dataset, the authors intend to conduct a further exhaustive examination of YOLO and other potential algorithms, enabling the refinement of the existing models and the establishment of new benchmarks for crack detection and segmentation. The anticipated results will significantly advance the field of infrastructure inspection, providing more accurate and reliable tools for real-time analysis and maintenance.

CONCLUSIONS

This study introduces YOLOv8 as a state-of-the-art solution for concrete crack detection. The comprehensive evaluation of various YOLOv8 models—nano, small, medium, large, and extra-large—employed a high-quality, diverse dataset thoroughly curated and annotated by Roboflow. An in-depth analysis of six distinct optimizers—SGD, Adam, AdamW, RMSprop, NAdam, and RAdam—was conducted alongside rigorous hyperparameter optimization. The results demonstrate that the YOLOv8m with the SGD optimizer model archives superior performance in both accuracy and speed, setting a new benchmark standard for optimal crack detection.

This research lays a robust foundation for integrating advanced computer vision techniques into infrastructure monitoring, facilitating more reliable, efficient, and proactive maintenance strategies for aging bridge networks. Future research is encouraged to explore generative modeling to enrich datasets and further benchmark YOLOv8 or other algorithms, aiming to define best practices particularly suited for real-time applications. This will help establish best practices for real-time applications and extend the benefits beyond bridge crack detection, leading to substantial advancements in various infrastructure monitoring tasks and improving the overall safety and reliability of transportation networks.

ACKNOWLEDGMENTS

Financial support to complete this work was provided in part by the U.S. Department of Transportation, Office of Assistant Secretary for Research and Technology under the auspices of Mid-America Transportation Center at the University of Nebraska, Lincoln (grant no. 00059709).

AUTHOR CONTRIBUTIONS

The authors confirm contribution to the paper as follows: study conception and design: G. Chen, G. Manogaran; data collection: G. Manogaran; analysis and interpretation of results: G. Manogaran, W. Z. Taffese, R. Sharma, M. H. Afsharmovahed; manuscript draft preparation: W. Z. Taffese, R. Sharma, M. H. Afsharmovahed. All authors reviewed the results and approved the final version of the manuscript.

REFERENCES

1. American Society of Civil Engineers. *ASCE's 2021 Infrastructure Report Card*. 2021.
2. *FEDERAL-AID POLICY GUIDE - SUBCHAPTER G - ENGINEERING AND TRAFFIC OPERATIONS*. 1994.
3. Kun, J., Z. Zhenhai, Y. Jiale, and D. Jianwu. A Deep Learning-based Method for Pixel-level Crack Detection on Concrete Bridges. *IET Image Processing*, Vol. 16, No. 10, 2022, pp. 2609–2622. <https://doi.org/10.1049/ipr2.12512>.
4. Li, G., T. Liu, Z. Fang, Q. Shen, and J. Ali. Automatic Bridge Crack Detection Using Boundary Refinement Based on Real-time Segmentation Network. *Structural Control and Health Monitoring*, Vol. 29, No. 9, 2022. <https://doi.org/10.1002/stc.2991>.

5. Li, R., J. Yu, F. Li, R. Yang, Y. Wang, and Z. Peng. Automatic Bridge Crack Detection Using Unmanned Aerial Vehicle and Faster R-CNN. *Construction and Building Materials*, Vol. 362, 2023, p. 129659. <https://doi.org/10.1016/j.conbuildmat.2022.129659>.
6. Lu, G., X. He, Q. Wang, F. Shao, J. Wang, and Q. Jiang. Bridge Crack Detection Based on Improved Single Shot Multi-Box Detector. *PLOS ONE*, Vol. 17, No. 10, 2022, p. e0275538. <https://doi.org/10.1371/journal.pone.0275538>.
7. He, M., and T. L. Lau. CrackHAM: A Novel Automatic Crack Detection Network Based on U-Net for Asphalt Pavement. *IEEE Access*, Vol. 12, 2024, pp. 12655–12666. <https://doi.org/10.1109/ACCESS.2024.3353729>.
8. Zoubir, H., M. Rguig, M. El Aroussi, R. Saadane, and A. Chehri. Pixel-Level Concrete Bridge Crack Detection Using Convolutional Neural Networks, Gabor Filters, and Attention Mechanisms. *Engineering Structures*, Vol. 314, 2024, p. 118343. <https://doi.org/10.1016/j.engstruct.2024.118343>.
9. Redmon, J., S. Divvala, R. Girshick, and A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. 2016.
10. Gong, H., J. Tešić, J. Tao, X. Luo, and F. Wang. Automated Pavement Crack Detection with Deep Learning Methods: What Are the Main Factors and How to Improve the Performance? *Transportation Research Record: Journal of the Transportation Research Board*, Vol. 2677, No. 10, 2023, pp. 311–323. <https://doi.org/10.1177/03611981231161358>.
11. Zhang, J., S. Qian, and C. Tan. Automated Bridge Crack Detection Method Based on Lightweight Vision Models. *Complex & Intelligent Systems*, Vol. 9, No. 2, 2023, pp. 1639–1652. <https://doi.org/10.1007/s40747-022-00876-6>.
12. Liu, Y., W. Gao, T. Zhao, Z. Wang, and Z. Wang. A Rapid Bridge Crack Detection Method Based on Deep Learning. *Applied Sciences*, Vol. 13, No. 17, 2023, p. 9878. <https://doi.org/10.3390/app13179878>.
13. Su, H., D. B. Kamanda, T. Han, C. Guo, R. Li, Z. Liu, F. Su, and L. Shang. Enhanced YOLO v3 for Precise Detection of Apparent Damage on Bridges amidst Complex Backgrounds. *Scientific Reports*, Vol. 14, No. 1, 2024, p. 8627. <https://doi.org/10.1038/s41598-024-58707-2>.
14. Yang, Y., L. Li, G. Yao, H. Du, Y. Chen, and L. Wu. An Modified Intelligent Real-Time Crack Detection Method for Bridge Based on Improved Target Detection Algorithm and Transfer Learning. *Frontiers in Materials*, Vol. 11, 2024. <https://doi.org/10.3389/fmats.2024.1351938>.
15. Xiong, C., T. Zayed, and E. M. Abdelkader. A Novel YOLOv8-GAM-Wise-IoU Model for Automated Detection of Bridge Surface Cracks. *Construction and Building Materials*, Vol. 414, 2024, p. 135025. <https://doi.org/10.1016/j.conbuildmat.2024.135025>.
16. Li, T., G. Liu, and S. Tan. Superficial Defect Detection for Concrete Bridges Using YOLOv8 with Attention Mechanism and Deformation Convolution. *Applied Sciences*, Vol. 14, No. 13, 2024, p. 5497. <https://doi.org/10.3390/app14135497>.
17. Matarneh, S., F. Elghaish, F. Pour Rahimian, E. Abdellatef, and S. Abrishami. Evaluation and Optimisation of Pre-Trained CNN Models for Asphalt Pavement Crack Detection and

- Classification. *Automation in Construction*, Vol. 160, 2024, p. 105297. <https://doi.org/10.1016/j.autcon.2024.105297>.
18. Zhang, Q., S. Chen, Y. Wu, Z. Ji, F. Yan, S. Huang, and Y. Liu. Improved U-Net Network Asphalt Pavement Crack Detection Method. *PLOS ONE*, Vol. 19, No. 5, 2024, p. e0300679. <https://doi.org/10.1371/journal.pone.0300679>.
 19. Qin, Z., Z. Li, Z. Zhang, Y. Bao, G. Yu, Y. Peng, and J. Sun. ThunderNet: Towards Real-Time Generic Object Detection. 2019.
 20. Su, P., H. Han, M. Liu, T. Yang, and S. Liu. MOD-YOLO: Rethinking the YOLO Architecture at the Level of Feature Information and Applying It to Crack Detection. *Expert Systems with Applications*, Vol. 237, 2024, p. 121346. <https://doi.org/10.1016/j.eswa.2023.121346>.
 21. Xiong, C., T. Zayed, X. Jiang, G. Alfalah, and E. M. Abelkader. A Novel Model for Instance Segmentation and Quantification of Bridge Surface Cracks—The YOLOv8-AFPN-MPD-IoU. *Sensors*, Vol. 24, No. 13, 2024, p. 4288. <https://doi.org/10.3390/s24134288>.
 22. Tang, G., C. Yin, X. Zhang, X. Liu, and S. Li. Crack-Detection Method for Asphalt Pavement Based on the Improved YOLOv5. *Journal of Performance of Constructed Facilities*, Vol. 38, No. 2, 2024. <https://doi.org/10.1061/JPCFEV.CFENG-4615>.
 23. Liu, Q., and Z. Liu. Asphalt Pavement Crack Detection Based on Improved YOLOv5s Algorithm. 2024.
 24. Tran, T. S., S. D. Nguyen, H. J. Lee, and V. P. Tran. Advanced Crack Detection and Segmentation on Bridge Decks Using Deep Learning. *Construction and Building Materials*, Vol. 400, 2023, p. 132839. <https://doi.org/10.1016/j.conbuildmat.2023.132839>.
 25. Dwyer, B. , N. J. , H. T. , et. al. Roboflow (Version 1.0) [Software]. Computer Vision.
 26. Glenn Jocher, A. C. J. Q. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>. Accessed Nov. 18, 2024.
 27. Li, C., L. Li, H. Jiang, K. Weng, Y. Geng, L. Li, Z. Ke, Q. Li, M. Cheng, W. Nie, Y. Li, B. Zhang, Y. Liang, L. Zhou, X. Xu, X. Chu, X. Wei, and X. Wei. YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications. 2022.
 28. Bochkovskiy, A., C.-Y. Wang, and H.-Y. M. Liao. YOLOv4: Optimal Speed and Accuracy of Object Detection. 2020.
 29. Wang, C.-Y., A. Bochkovskiy, and H.-Y. M. Liao. YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors. 2022.
 30. Redmon, J., and A. Farhadi. YOLOv3: An Incremental Improvement. 2018.
 31. Redmon, J., and A. Farhadi. YOLO9000: Better, Faster, Stronger. 2017.
 32. Ramos, L., E. Casas, E. Bendek, C. Romero, and F. Rivas-Echeverría. Hyperparameter Optimization of YOLOv8 for Smoke and Wildfire Detection: Implications for Agricultural and Environmental Safety. *Artificial Intelligence in Agriculture*, Vol. 12, 2024, pp. 109–126. <https://doi.org/10.1016/j.aiia.2024.05.003>.

33. Chen, X., C. Yuan, and Y. Liu. Research On Bird Nest Detection Method of Transmission Line Based on YOLOv8. *International Journal of Computer Science and Information Technology*, Vol. 3, No. 2, 2024, pp. 336–344. <https://doi.org/10.62051/ijcsit.v3n2.36>.
34. Robbins, H., and S. Monro. *A Stochastic Approximation Method*. 1951.
35. Kingma, D. P., and J. Lei Ba. ADAM: A Method for Stochastic Optimization. Presented at 3rd International Conference for Learning Representations, San Diego, 2015.
36. Loshchilov, I., and F. Hutter. Decoupled Weight Decay Regularization. Presented at the 9th International Conference for Learning Representations, New Orleans, 2019.
37. Kurbiel, T., and S. Khaleghian. Training of Deep Neural Networks Based on Distance Measures Using RMSProp. 2017.
38. Liu, L., G. Tech, P. He, W. Chen, X. Liu, J. Gao, and J. Han. On the Variance of the Adaptive Learning Rate and Beyond. Presented at the 10th International Conference for Learning Representations, 2020.
39. Timothy Dozat. Incorporating Nesterov Momentum into Adam. Presented at 4th International Conference for Learning Representations, San Juan, 2016.
40. Czitrom, V. One-Factor-at-a-Time versus Designed Experiments. *The American Statistician*, Vol. 53, No. 2, 1999, pp. 126–131. <https://doi.org/10.1080/00031305.1999.10474445>.
41. Taffese, W. Z., and L. Espinosa-Leal. Prediction of Chloride Resistance Level of Concrete Using Machine Learning for Durability and Service Life Assessment of Building Structures. *Journal of Building Engineering*, Vol. 60, 2022, p. 105146. <https://doi.org/10.1016/j.jobbe.2022.105146>.
42. Karras, T., S. Laine, and T. Aila. A Style-Based Generator Architecture for Generative Adversarial Networks. 2018.
43. Dosovitskiy, A., J. T. Springenberg, M. Tatarchenko, and T. Brox. Learning to Generate Chairs, Tables and Cars with Convolutional Networks. 2014.
44. Kingma, D. P., and M. Welling. Auto-Encoding Variational Bayes. 2013.